Desafio — o laço do agente: percepção → decisão → ação, à mão

D2 — Fundamentos de Agentes Inteligentes · desafio opcional (o pacote D2-09) Sem nota · sem prazo · a solução comentada viaja junto

> **Onde trabalhar:** extraia o zip na raiz do SEU repositório e ative o ambiente antes de rodar — Windows, na raiz: `.venv\Scripts\activate.bat`. Renomeie `esqueleto.py` para **`resposta.py`** — é desse nome que o teste importa.

O que é um desafio

Material **opcional**: sem nota, sem peso na correção, sem prazo além do que você quiser se dar. A solução comentada vem junto — abrir antes de tentar é escolha sua, não falha.

Por que este desafio existe

As duas demonstrações mostraram o laço: contexto montado → chamada ao modelo → pedido de ferramenta → o programa executa → observação → repete. Este desafio é escrever esse laço com as próprias mãos, **sem arcabouço pronto** — porque é exatamente a peça que a D10 vai construir com framework, e quem escreveu o laço à mão reconhece o que o framework esconde. Sem nota e sem prazo: existe para quem quer ver o mecanismo por dentro.

As duas exigências de mecânica (as que separam 2026 de 2023)

1. **A chamada de ferramenta é no formato estruturado nativo do modelo** — `functionDeclarations` no REST do Gemini. Nada de instruir o modelo a escrever "Ação:" e caçar a palavra seguinte com expressão regular: o padrão de 2023 é frágil e não se usa mais. O princípio dele sobreviveu; a implementação, não.
2. **A execução é manual.** O modelo devolve um pedido de função; QUEM executa é o seu laço. No REST direto não existe executor automático — se você optar por um SDK no seu projeto, descubra e desligue o executor automático dele, ou o desafio perde a razão de existir.

E a terceira, do bloco B3 do último sábado: **teto de rodadas**, com mensagem clara ao atingi-lo. Deliberar sem teto é irracional — pensar custa tokens, tempo e dinheiro, e o pior caso sempre chega.

O que fazer

Com as duas ferramentas das demonstrações (uma de **consulta** e uma de **escrita** — as mesmas, para você reconhecer o que viu na janela do E4):

1. **TODO 1** — o registro de ferramentas no formato nativo: as duas declarações, com `name`, `description` e `parameters`.
2. **TODO 2** — a chamada REST (`generateContent`) e a tradução da resposta: parte com `functionCall` → pedido; partes de texto → resposta final. Cuidado que se aprende com 400 na mão: a parte do modelo

volta INTEIRA na mensagem seguinte — os modelos com raciocínio exigem o `thoughtSignature` de volta.

3. **TODO 3** — o laço: executar a ferramenta pedida, devolver o retorno como `functionResponse`, repetir — até texto final, ou até o **teto**.
4. Rode o modo **sem rede** — zero requisições, zero cota:

```

python resposta.py --sem-rede python -m pytest teste.py -q

```

1. **Encene o seu laço** — o rastro que o desafio grava está no MESMO formato das demonstrações, e o mesmo `encenar.py` o apresenta:

```

python encenar.py rastro-do-laco.json

```

É o arco da disciplina fechando: o laço que você escreveu, contado na linguagem dos quatro encontros.

A chave é sua, e não está em lugar nenhum

O modo real usa a SUA chave (`GEMINI_API_KEY` no ambiente), a mesma da D1 — o material de lá ensina a criar; aqui não se repete. Nenhuma chave entra em arquivo do projeto: nem a sua, nem de exemplo, nem mascarada. O programa falha com mensagem clara se a chave faltar — e o `teste.py` nunca exige rede: ele testa o laço contra um roteiro gravado; a chamada real é sua, quando você quiser.

Critério de pronto

```
python resposta.py --sem-rede
python -m pytest teste.py -q
python encenar.py rastro-do-laco.json
```

Os três rodando — e você sabendo explicar cada linha do laço: onde o modelo decide, onde o programa executa, e onde o teto segura.

Se travar

1. **As declarações de ferramenta não passam?** Cada uma precisa de `name` (letra no início, sem acento), `description` e `parameters` com tipos em maiúsculas (`OBJECT`, `STRING`). O erro 400 do modelo cita a declaração que quebrou — leia a mensagem até o fim.
2. **A segunda rodada falha com 400?** A parte do modelo voltou recomposta e perdeu o `thoughtSignature` — devolva a parte INTEIRA, como veio.
3. **O laço não para?** O teste do teto existe exatamente para isso: roteiro que nunca responde, limite em 4 — se ele não segurar, o laço não tem condição de parada.

4. **Quiser ver o gabarito?** O `solucao.py` é a solução completa e comentada, com o modo real e o modo sem rede — nossa autorização expressa para abrir, executar e explicar. Copiar sem entender cobra o preço em D10, quando a mesma peça voltar com framework por cima.
